

SaaS Factures IA

Liste des Améliorations Prioritaires

Analyse technique du code source · Recommandations d'amélioration

Généré le 18 May 2026

Rapport d'analyse technique

Ce document liste les **10 améliorations prioritaires** identifiées après analyse complète du code source du projet SaaS Factures IA. Chaque amélioration inclut le contexte du problème, le code à modifier et les étapes de correction.

2	4	3	1
Corrections critiques (bloquantes en prod)	Corrections haute priorité	Améliorations moyennes	Amélioration basse priorité

Vue d'ensemble — Toutes les améliorations

N°	Amélioration	Fichier(s)	Priorité	Statut actuel
1	Corriger TESSERACT_CMD pour le déploiement Linux	backend/config/settings.py	CRITIQUE	À corriger
2	Passer TESSERACT_LANG de 'eng' à 'fra+eng'	backend/.env.example	HAUTE	À corriger
3	Sécuriser le webhook Stripe — supprimer le fallback dan	backend/invoices/stripe_views.py	CRITIQUE	À corriger
4	Corriger le calcul de confiance dans extracted_data	backend/invoices/services.py	HAUTE	À corriger
5	Corriger quota_remaining() — compter uniquement le mois	backend/invoices/models.py	HAUTE	À corriger
6	Compléter requirements.txt — pypdf et Pillow manquants	backend/requirements.txt	HAUTE	À corriger
7	Implémenter le Stripe Portal (gestion d'abonnement)	backend/invoices/stripe_views.py	MOYENNE	À corriger
8	Enrichir la couverture de tests	backend/invoices/tests.py	MOYENNE	À corriger
9	Ajouter un Dockerfile pour la production	backend/Dockerfile (à créer)	MOYENNE	À corriger
10	Migrer de SQLite vers PostgreSQL	backend/config/settings.py	BASSE	À corriger

Légende des priorités :

CRITIQUE	Bloque la mise en production ou crée une faille de sécurité
HAUTE	Impact fonctionnel direct — données incorrectes ou fonctionnalité cassée
MOYENNE	Améliore la robustesse et l'expérience utilisateur
BASSE	Bonne pratique — à implémenter avant la mise en production finale

1 Corriger TESSERACT_CMD pour le déploiement Linux

■ Fichier(s)	backend/config/settings.py + .env.example
■ Priorité	CRITIQUE
■ Impact	<i>Bloque totalement le déploiement sur Railway / tout serveur Linux</i>

■ Problème identifié

Le chemin par défaut pointe vers Windows :

```
TESSERACT_CMD = os.getenv("TESSERACT_CMD", r"C:\Program  
Files\Tesseract-OCR\tesseract.exe")
```

En production Linux (Railway, Render, VPS), ce chemin n'existe pas.

Résultat : l'OCR échoue silencieusement sur toutes les factures.

■ Solution proposée

```
# settings.py – corriger le default Linux :
```

```
TESSERACT_CMD = os.getenv("TESSERACT_CMD", "/usr/bin/tesseract")
```

```
# .env.example – documenter les deux OS :
```

```
# Linux/Mac : /usr/bin/tesseract
```

```
# Windows : C:\Program Files\Tesseract-OCR\tesseract.exe
```

```
TESSERACT_CMD=/usr/bin/tesseract
```

```
# Railway : ajouter dans le Dockerfile ou Procfile :
```

```
# apt-get install -y tesseract-ocr tesseract-ocr-fra
```

■ Notes & étapes suivantes

- Installer le pack de langue française sur le serveur : `apt-get install tesseract-ocr-fra`
- Vérifier la disponibilité via la fonction `tesseract_is_available()` déjà présente dans `services.py`

2 Passer TESSERACT_LANG de 'eng' à 'fra+eng'

■ Fichier(s)	backend/.env.example + backend/invoices/services.py
■ Priorité	HAUTE
■ Impact	OCR défaillant sur toutes les factures en français (accents, €, libellés)

■ Problème identifié

```
# .env.example actuel :
```

```
TESSERACT_LANG=eng
```

Avec eng uniquement, Tesseract échoue sur les caractères :

é à ù ê ç î → remplacés par des points d'interrogation ou omis.

Le symbole € est souvent converti en E ou ignoré.

Les mots 'Facture', 'Fournisseur', 'Total TTC' sont mal reconnus.

■ Solution proposée

```
# .env.example – changer la langue :
```

```
TESSERACT_LANG=fra+eng
```

```
# services.py – améliorer le preprocessing OCR :
```

```
def ocr_image(image):
```

```
    import numpy as np
```

```
    grayscale = image.convert('L')
```

```
    arr = np.array(grayscale)
```

```
    threshold = arr.mean() - arr.std() * 0.5 # seuil adaptatif
```

```
    boosted = grayscale.point(lambda px: 0 if px < threshold else 255)
```

```
    return pytesseract.image_to_string(
```

```
        boosted, lang=tesseract_languages(),
```

```
        config="--oem 3 --psm 6"
```

```
    )
```

■ Notes & étapes suivantes

- fra+eng permet de reconnaître les deux langues simultanément
- Le seuil adaptatif (basé sur moyenne - écart-type) est plus robuste que le seuil fixe à 180
- Ajouter numpy>=1.26 dans requirements.txt si le seuil adaptatif est utilisé

3 Sécuriser le webhook Stripe — supprimer le fallback dangereux

■ Fichier(s)	backend/invoices/stripe_views.py
■ Priorité	CRITIQUE
■ Impact	<i>N'importe qui peut forger un faux webhook et activer un plan payant sans payer</i>

■ Problème identifié

```
# stripe_views.py – ligne ~30 :  
  
except Exception as e:  
  
# Handle cases where secret is not configured  
  
event = json.loads(payload) # ← FAILLE DE SÉCURITÉ
```

Si STRIPE_WEBHOOK_SECRET n'est pas configuré, la signature n'est PAS vérifiée. Un attaquant peut envoyer un POST forgé avec event type 'checkout.session.completed' et activer n'importe quel plan Enterprise gratuitement.

■ Solution proposée

```
# stripe_views.py – version sécurisée :  
  
def post(self, request):  
  
    if not settings.STRIPE_WEBHOOK_SECRET:  
  
        return HttpResponse(status=400) # Refuser si non configuré  
  
    try:  
  
        event = stripe.Webhook.construct_event(  
  
            request.body,  
  
            request.META.get('HTTP_STRIPE_SIGNATURE'),  
  
            settings.STRIPE_WEBHOOK_SECRET  
  
        )  
  
    except (ValueError, stripe.error.SignatureVerificationError):  
  
        return HttpResponse(status=400)  
  
    # Traiter l'événement normalement...
```

■ Notes & étapes suivantes

- En développement sans Stripe configuré, désactiver l'endpoint plutôt que le rendre non sécurisé
 - Ajouter STRIPE_WEBHOOK_SECRET dans la checklist de déploiement Railway
 - Tester le webhook avec la CLI Stripe : `stripe listen --forward-to localhost:8000/api/stripe/webhook/`
-

4 Corriger le calcul de confiance dans `extracted_data`

■ Fichier(s)	backend/invoices/services.py
■ Priorité	HAUTE
■ Impact	Affiche 0.95% au lieu de 95% — indicateur de confiance inutilisable

■ Problème identifié

```
# services.py – double division par 100 :  
  
invoice.confidence = extracted['confidence'] # ex: 0.95 (de Claude API)  
  
# ...
```

'supplier': {'value': ..., 'confidence': float(invoice.confidence) / 100},

```
# Résultat : 0.95 / 100 = 0.0095 ← affiché comme 0.95% au lieu de 95%
```

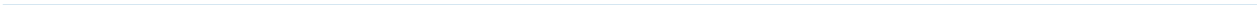
Si Claude retourne 95 (pas 0.95), on obtient 0.95 en base,
puis 0.0095 dans `extracted_data`. L'UI reçoit des valeurs incohérentes.

■ Solution proposée

```
# Dans anthropic_extract() – normaliser à l'entrée (0.0 à 1.0) :  
  
raw_conf = parsed.get('confidence', 0.8)  
  
# Si Claude retourne 95, diviser par 100. Si 0.95, garder tel quel.  
  
if float(raw_conf) > 1.0:  
    raw_conf = float(raw_conf) / 100  
  
parsed['confidence'] = Decimal(str(round(float(raw_conf), 4)))  
  
# Dans extracted_data – ne plus diviser :  
  
invoice.extracted_data = {  
    'supplier': {'value': invoice.supplier,  
        'confidence': float(invoice.confidence)}, # ex: 0.95  
    ...  
}
```

■ Notes & étapes suivantes

- Standardiser : la valeur confiance est toujours entre 0.0 et 1.0 en base de données
- Le frontend affiche : `Math.round(confidence * 100) + '%'`
- La valeur heuristique est déjà correcte à 72.00 → à convertir en 0.72 aussi



5 Corriger quota_remaining() — compter uniquement le mois courant

■ Fichier(s)	backend/invoices/models.py
■ Priorité	HAUTE
■ Impact	Un utilisateur avec 25 factures historiques voit un quota négatif dès le 1er du mois

■ Problème identifié

```
# models.py – actuel :  
  
def invoice_count(self):  
  
    return self.invoices.count() # ← TOUTES les factures, tous mois confondus  
  
def quota_remaining(self):  
  
    return max(self.monthly_quota - self.invoice_count(), 0)
```

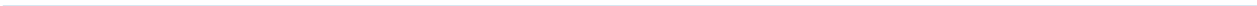
Problème : un utilisateur Free avec 25 factures au total
(quota = 20/mois) voit quota_remaining() = 0 en permanence,
même en début de nouveau mois.

■ Solution proposée

```
# models.py – version corrigée :  
  
from django.utils import timezone  
  
def invoice_count_this_month(self):  
  
    now = timezone.now()  
  
    return self.invoices.filter(  
  
        uploaded_at__year=now.year,  
  
        uploaded_at__month=now.month  
  
    ).count()  
  
def quota_remaining(self):  
  
    return max(self.monthly_quota - self.invoice_count_this_month(), 0)
```

■ Notes & étapes suivantes

- Mettre à jour le payload dashboard dans views.py (_summary_payload) pour utiliser invoice_count_this_month()
- Ajouter un test unitaire : créer 21 factures le mois précédent, vérifier quota_remaining() = 20 en début du mois suivant
- Exposer quotaUsedThisMonth dans le payload JSON pour le frontend



6 Compléter requirements.txt — pypdf et Pillow manquants

■ Fichier(s)	backend/requirements.txt
■ Priorité	HAUTE
■ Impact	Le déploiement <code>pip install -r requirements.txt</code> échoue — app non fonctionnelle

■ Problème identifié

```
# requirements.txt actuel (extrait) :
```

```
Django>=5.2,<6.1
```

```
djangoorestframework>=3.17,<3.18
```

```
boto3>=1.43,<1.44
```

```
pytesseract>=0.3.13,<0.4
```

```
pdf2image>=1.17,<1.18
```

```
# ← pypdf ABSENT alors qu'il est importé dans services.py
```

```
# ← Pillow ABSENT alors qu'il est utilisé pour Image.open()
```

■ Solution proposée

```
# requirements.txt – version complète :
```

```
Django>=5.2,<6.1
```

```
djangoorestframework>=3.17,<3.18
```

```
djangoorestframework-simplejwt>=5.5,<5.6
```

```
boto3>=1.43,<1.44
```

```
pytesseract>=0.3.13,<0.4
```

```
pdf2image>=1.17,<1.18
```

```
pypdf>=4.0,<5.0 # ← AJOUTER
```

```
Pillow>=10.0,<11.0 # ← AJOUTER
```

```
celery>=5.6,<5.7
```

```
redis>=7.4,<7.5
```

```
stripe>=11.0.0
```

```
reportlab>=4.0.0
```

■ Notes & étapes suivantes

- Vérifier la compatibilité Pillow 10 avec pdf2image (certaines versions nécessitent Pillow 9)

- Ajouter aussi numpy si le seuil adaptatif OCR est implémenté (amélioration n°2)
 - Utiliser pip freeze > requirements.txt en fin de dev pour capturer les versions exactes
-

7 Implémenter le Stripe Portal (gestion d'abonnement)

■ Fichier(s)	backend/invoices/stripe_views.py + frontend/src/App.jsx
■ Priorité	MOYENNE
■ Impact	L'utilisateur ne peut pas annuler ou changer de plan — expérience SaaS incomplète

■ Problème identifié

```
# Actuellement, stripe_views.py n'expose pas de Stripe Billing Portal.

# L'utilisateur ne peut que s'abonner, pas gérer son abonnement.

# Sans Portal, l'annulation est impossible → risque de litiges carte bancaire.
```

■ Solution proposée

```
# stripe_views.py – ajouter PortalSessionView :

class CreatePortalSessionView(APIView):

    permission_classes = [permissions.IsAuthenticated, IsOwner]

    def post(self, request):

        org = request.user.organization

        if not org.stripe_customer_id:

            return Response({'error': 'Pas de client Stripe'}, status=400)

        session = stripe.billing_portal.Session.create(

            customer=org.stripe_customer_id,

            return_url=settings.FRONTEND_URL + '/dashboard',

        )

        return Response({'url': session.url})

# urls.py – ajouter :

path('stripe/portal/', CreatePortalSessionView.as_view()),
```

■ Notes & étapes suivantes

- Activer le Stripe Billing Portal dans le dashboard Stripe (Settings > Billing > Customer portal)
- Côté React : bouton 'Gérer mon abonnement' → POST /api/stripe/portal/ → redirect vers session.url
- Stocker stripe_customer_id dès la création du checkout : récupérer dans le webhook checkout.session.completed

8 Enrichir la couverture de tests

■ Fichier(s)	backend/invoices/tests.py
■ Priorité	MOYENNE
■ Impact	3 tests seulement — impossible de détecter des régressions lors d'évolutions

■ Problème identifié

```
# tests.py actuel : 3 tests couvrant uniquement :  
  
# - quota_remaining() (ModelTests)  
  
# - register crée org + owner (APITests)  
  
# - accès inter-organisation refusé (InvoicePermissionsTests)  
  
# Non testés : pipeline OCR, extraction IA, validation,  
  
# export CSV/PDF, quotas mensuels, Stripe webhooks.
```

■ Solution proposée

```
# tests.py – ajouter (avec unittest.mock) :  
  
from unittest.mock import patch, MagicMock  
  
class PipelineTests(TestCase):  
  
    @patch('invoices.services.anthropic_extract')  
  
    @patch('invoices.services.extract_text_from_invoice')  
  
    def test_process_invoice_with_mock_ai(self, mock_ocr, mock_ai):  
  
        mock_ocr.return_value = ('Facture Orange SA\nTTC: 45.00', 'pypdf')  
  
        mock_ai.return_value = {  
  
            'supplier': 'Orange SA', 'invoiceNumber': 'F-2026-001',  
  
            'invoiceDate': date(2026, 5, 1), 'amountHt': Decimal('37.50'),  
  
            'amountTva': Decimal('7.50'), 'amountTtc': Decimal('45.00'),  
  
            'category': 'Telecom', 'confidence': Decimal('0.95')  
  
        }  
  
        invoice = Invoice.objects.create(org=self.org, file_name='test.pdf')  
  
        process_invoice(invoice)
```



```
invoice.refresh_from_db()
```

```
self.assertEqual(invoice.status, Invoice.STATUS_REVIEW)
```

```
self.assertEqual(invoice.supplier, 'Orange SA')
```

■ Notes & étapes suivantes

- Objectif minimum : 80% de couverture sur services.py et views.py
 - Configurer pytest-django et pytest-cov dans requirements-dev.txt
 - Ajouter le rapport de couverture dans GitHub Actions CI
-

9 Ajouter un Dockerfile pour la production

■ Fichier(s)	backend/Dockerfile (à créer)
■ Priorité	MOYENNE
■ Impact	Déploiement Railway non reproductible — TESSERACT et Pillow peuvent ne pas s'installer

■ Problème identifié

```
# Actuellement, aucun Dockerfile dans le projet.

# Railway détecte Django et installe via requirements.txt,

# mais Tesseract OCR est un binaire système, pas un package pip.

# Sans Dockerfile, Tesseract n'est pas installé sur le serveur.
```

■ Solution proposée

```
# backend/Dockerfile :

FROM python:3.12-slim

RUN apt-get update && apt-get install -y --no-install-recommends \
tesseract-ocr tesseract-ocr-fra tesseract-ocr-eng \
poppler-utils libpq-dev gcc \
&& rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

ENV TESSERACT_CMD=/usr/bin/tesseract

ENV TESSERACT_LANG=fra+eng

CMD ["unicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000"]
```

■ Notes & étapes suivantes

- Ajouter unicorn dans requirements.txt pour la production
- Configurer railway.toml pour pointer sur ce Dockerfile
- Séparer Dockerfile.worker pour le Celery worker (même image, CMD différente)

10 Migrer de SQLite vers PostgreSQL

■ Fichier(s)	backend/config/settings.py + backend/requirements.txt
■ Priorité	BASSE
■ Impact	SQLite ne supporte pas les accès concurrents — plusieurs requêtes simultanées peuvent corrompre les données

■ Problème identifié

```
# settings.py actuel :  
  
DATABASES = {  
  
    'default': {  
  
        'ENGINE': 'django.db.backends.sqlite3',  
  
        'NAME': BASE_DIR / 'db.sqlite3',  
  
    }  
}  
  
# SQLite est acceptable en développement local,  
  
# mais Railway propose PostgreSQL nativement.  
  
# Avec Celery + Django en parallèle, SQLite génère des erreurs 'database is  
locked'.
```

■ Solution proposée

```
# settings.py – version PostgreSQL :  
  
import dj_database_url  
  
DATABASES = {  
  
    'default': dj_database_url.config(  
  
        default=os.getenv('DATABASE_URL', 'sqlite:///db.sqlite3'),  
  
        conn_max_age=600  
  
    )  
  
}  
  
# requirements.txt – ajouter :  
  
dj-database-url>=2.0,<3.0  
  
psycopg2-binary>=2.9,<3.0
```

```
# .env.example – ajouter :
```

```
DATABASE_URL=postgresql://user:password@localhost:5432/saas_factures
```

■ Notes & étapes suivantes

- Railway crée automatiquement une variable DATABASE_URL lors de l'ajout d'un plugin PostgreSQL
 - Garder SQLite comme fallback (valeur par défaut) pour le développement local sans config
 - Exécuter migrate après le changement : `python manage.py migrate`
-

Plan d'action recommandé

Semaine	Actions	Améliorations
S1 (Urgent)	Corrections bloquantes avant tout déploiement	#1 TESSERACT_CMD Linux #3 Webhook Stripe sécurisé #6 requirements.txt complet
S2 (Prioritaire)	Corrections fonctionnelles critiques	#2 Langue OCR fra+eng #4 Confiance normalisée #5 Quota mensuel corrigé
S3 (Amélioration)	Fonctionnalités manquantes + infrastructure	#7 Stripe Portal #9 Dockerfile production
S4 (Qualité)	Tests et base de données production	#8 Tests unitaires pipeline #10 Migration PostgreSQL

En appliquant ces 10 améliorations, le projet passe d'un MVP fonctionnel à un produit déployable en production, sécurisé, et prêt à accueillir de vrais utilisateurs payants.